Argo

Banco de dados

Diagrama de relacionamentos e arquitetura de dados

Copyright (©) 2026 Erik Pimenta

Versão 1.1.0 · Outubro de 2026 · Licença MIT

Sumário

- **Visão geral** 4
  - Um banco por serviço 4
  - Banco suportado 5
- **Diagrama de relacionamentos** 6
  - Identidade · argo_identity 7
  - Catálogo · argo_catalog 8
  - Pedidos · argo_orders 9
  - Faturamento · argo_billing 10
- **Regras e convenções** 11
  - Convenções do modelo 11
  - Integridade e concorrência 11
  - Tipos usados 12
- **Tabelas** 13
  - users · argo_identity 13
  - refresh_tokens · argo_identity 13
  - products · argo_catalog 14
  - stock_movements · argo_catalog 15
  - orders · argo_orders 15
  - order_items · argo_orders 16
  - order_status_history · argo_orders 17
  - invoices · argo_billing 17
  - invoice_items · argo_billing 18
  - invoice_number_seq · argo_billing 18
- **Fluxo de situações do pedido** 19
- **Como parear o banco** 20
  - Com Docker (recomendado) 20
  - MySQL próprio 20
  - Banco gerenciado 20
  - Backup e restauração 21

- Problemas comuns 21
- **Evolução do esquema** 22

Capítulo 1

Visão geral

Este documento descreve a arquitetura de dados que o Argo segue: quatro bancos MySQL, um por serviço, com 10 tabelas no total. Mostra os relacionamentos, todos os campos com tipos e chaves, as regras de cada tabela e como preparar o banco para usar o projeto.

Um banco por serviço

Cada microserviço é dono dos próprios dados e nenhum lê o banco de outro. Os quatro bancos podem ficar no mesmo servidor MySQL (como no Docker Compose) ou em servidores diferentes, sem mudar código: cada serviço recebe só a sua URL de conexão.

Banco	Serviço	Tecnologia	Tabelas
`argo_identity`	Identidade	Python · Django · ORM e migrações do Django	`users`, `refresh_tokens`
`argo_catalog`	Catálogo e estoque	Python · Django · ORM e migrações do Django	`products`, `stock_movements`
`argo_orders`	Pedidos	Python · Django · ORM e migrações do Django	`orders`, `order_items`, `order_status_history`
`argo_billing`	Faturamento	Java · Spring Boot · JPA · Flyway	`invoices`, `invoice_items`, `invoice_number_seq`

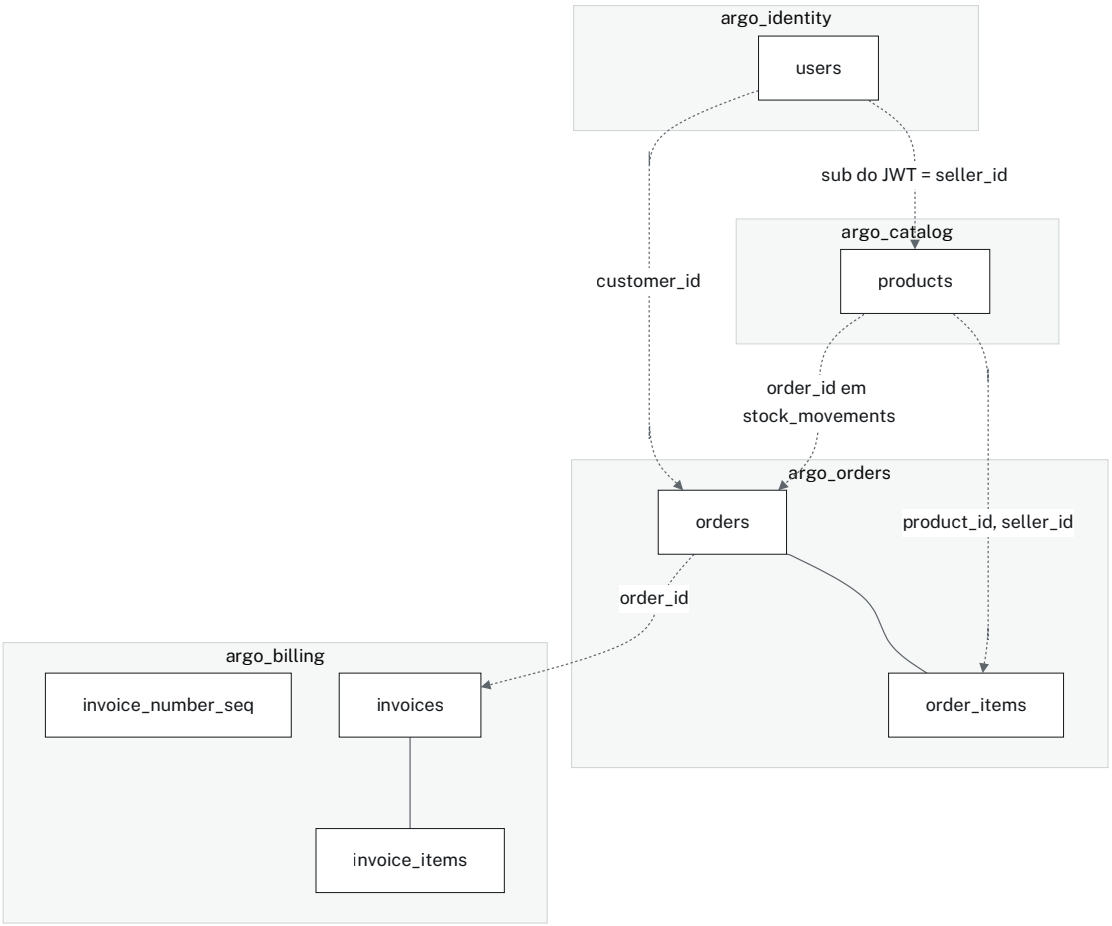
Banco suportado

Item	Valor
MySQL	8.4 LTS (imagem `mysql:8.4`), o mínimo do Django 6.1; tabelas InnoDB
Driver (Python)	mysqlclient 2.3, pelo backend MySQL do Django
Driver (Java)	MySQL Connector/J, com o Flyway (flyway-mysql)
Codificação	utf8mb4 com a collation utf8mb4_0900_ai_ci (comparações de texto não diferenciam maiúsculas nem acentos)
Fuso	Todas as datas em `DATETIME(6)`, gravadas em UTC (`USE_TZ = True` no Django; `hibernate.jdbc.time_zone = UTC` no Java)
Isolamento	READ COMMITTED (padrão do Django no MySQL)
Criação das tabelas (Python)	Migrações do Django em `backend/argo/<serviço>/migrations/`, aplicadas pelo `python manage.py serve` na subida; o SQL equivalente está em `database/argo_*.sql`
Criação das tabelas (Java)	Flyway aplica `V1__invoices.sql`; o Hibernate só valida (`ddl-auto: validate`)
Testes	pytest-django cria bancos SQLite temporários (ou MySQL, se configurado); H2 em modo MySQL no Java

Capítulo 2

Diagrama de relacionamentos

Dentro de cada banco os relacionamentos são chaves estrangeiras de verdade. Entre bancos, a ligação é feita por identificadores copiados (linhas tracejadas abaixo), sem chave estrangeira, porque cada serviço pode estar em outro servidor.



Ligações lógicas entre os bancos

Identidade · argo_identity

users		
varchar	id	PK
varchar	email	UK
varchar	password_hash	
varchar	role	
bool	active	
int	failed_attempts	
datetime	locked_until	

tem sessões

refresh_tokens		
varchar	id	PK
varchar	user_id	FK
varchar	family_id	
varchar	token_hash	UK
datetime	expires_at	
datetime	revoked_at	
varchar	replaced_by	

users 1 → N refresh_tokens

Catálogo · argo_catalog

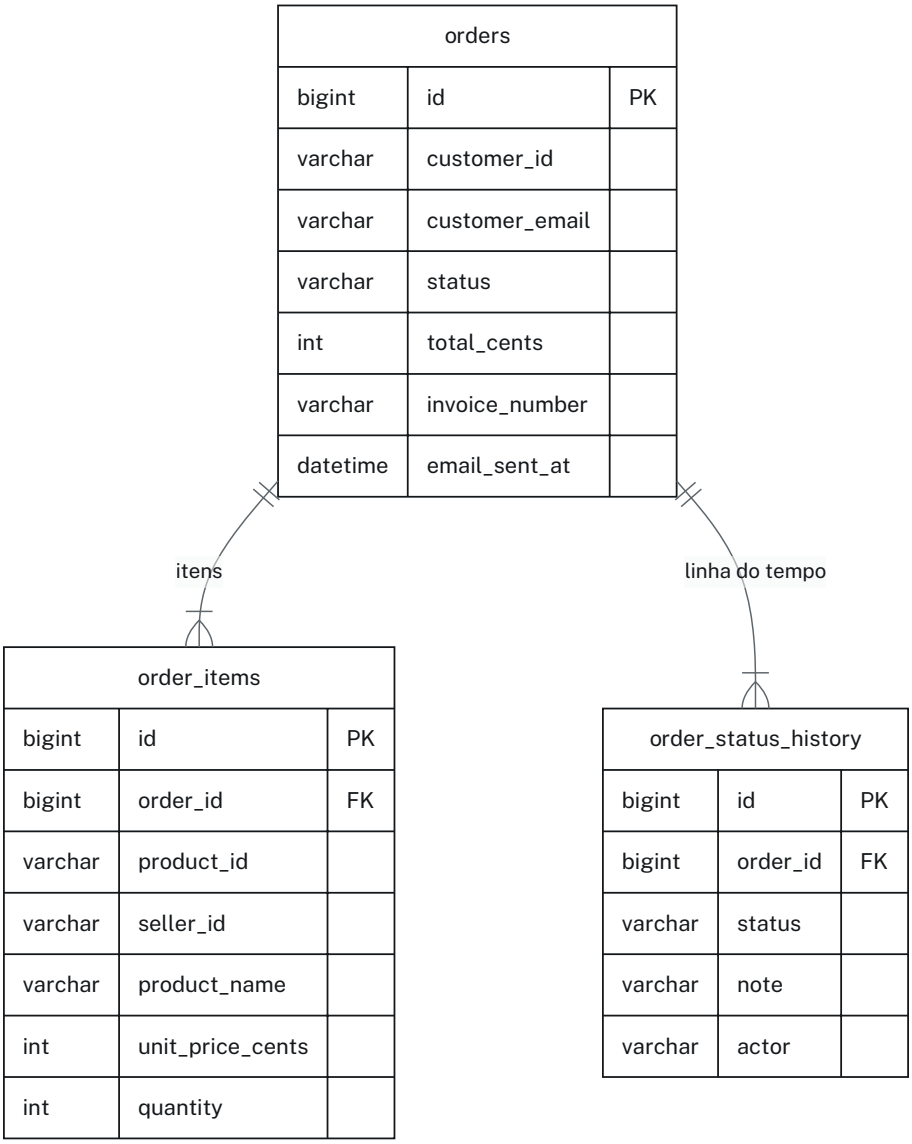
products		
varchar	id	PK
varchar	seller_id	
varchar	sku	UK
varchar	name	
int	price_cents	
int	stock	
int	low_stock_threshold	
bool	active	
int	version	

histórico

stock_movements		
bigint	id	PK
varchar	product_id	FK
varchar	kind	
int	delta	
int	stock_after	
bigint	order_id	
varchar	actor_id	

products 1 → N stock_movements

Pedidos · argo_orders



orders 1 → N order_items e order_status_history

Faturamento · argo_billing

invoices		
bigint	id	PK
varchar	number	UK
bigint	order_id	UK
varchar	customer_id	
varchar	status	
bigint	total_cents	
bigint	tax_cents	
decimal	tax_rate	

itens

invoice_items		
bigint	id	PK
bigint	invoice_id	FK
varchar	product_id	
varchar	seller_id	
int	quantity	
bigint	unit_price_cents	

invoice_number_seq		
tinyint	id	PK
bigint	next_val	

invoices 1 → N invoice_items

Capítulo 3

Regras e convenções

As regras abaixo valem para os quatro bancos e devem ser mantidas por qualquer mudança no esquema.

Convenções do modelo

- **Identificadores:** contas e produtos usam UUID em texto (`VARCHAR(36)`), que circula entre serviços e no JWT. Pedidos, movimentos e notas usam `BIGINT` sequencial (`BigAutoField` do Django e IDENTITY do JPA, coluna AUTO_INCREMENT).
- **Dinheiro em centavos:** todo valor é inteiro (`*_cents`). Nada de ponto flutuante em preço.
- **Datas sempre em UTC** (`DATETIME(6)` , sem fuso na coluna); a interface converte para o horário de Brasília.
- **Cópia do momento:** o pedido guarda nome, SKU, preço e vendedor de cada item. Mudar o produto depois não altera pedidos antigos.
- **Exclusão lógica:** produtos são desativados (`active = false`), nunca apagados, para não quebrar o histórico.
- **Nomes:** tabelas no plural em inglês (`db_table` fixo em cada modelo), colunas em `snake_case` , restrições e índices declarados com prefixo `uq_` , `ck_` e `ix_` . Chaves estrangeiras criadas pelo próprio Django usam o nome automático dele.

Integridade e concorrência

- **CHECKs** impedem estoque negativo, preço zero, quantidade zero e situações fora da lista. Eles ficam no `Meta.constraints` de cada modelo e viram SQL pelas migrações. O MySQL aplica CHECK desde a versão 8.0.16.
- **Baixa de estoque** com `select_for_update()` (`SELECT ... FOR UPDATE`) dentro de `transaction.atomic()` , sempre em ordem de id para evitar deadlock entre dois pedidos.
- **Idempotência** garantida pelo banco: `uq_stock_movements_order_product_kind` impede baixar ou devolver duas vezes o mesmo pedido; `uq_invoices_order` impede duas notas para o mesmo pedido.
- **Numeração das notas sem buracos:** o MySQL não tem SEQUENCE, então `invoice_number_seq` guarda o próximo número numa linha só, travada com `SELECT ... FOR UPDATE` na mesma transação que grava a nota.
- **Versão otimista** em `products.version` : a edição envia a versão lida e recebe 409 se outra pessoa alterou antes.
- **ON DELETE CASCADE no banco**: as chaves estrangeiras usam `DB_CASCADE` do Django 6.1, então a regra vale também para quem mexe direto no SQL.
- **Segredos nunca em texto:** senha no hasher bcrypt do Django (`bcrypt_sha256`), refresh token em SHA-256.

Tipos usados

Tipo	Onde	Motivo
`VARCHAR(36)`	UUIDs	Mesmo formato no Django, no Java e no JWT
`BIGINT AUTO_INCREMENT` / `BIGINT`	Sequenciais	Volume alto de pedidos e movimentos
`INTEGER` / `BIGINT`	Centavos e quantidades	Precisão exata
`DECIMAL(5,4)`	`tax_rate`	Alíquota exata, ex.: 0,1645
`DATETIME(6)`	Datas	Sempre em UTC, com microssegundos (o TIMESTAMP do MySQL termina em 2038)
`BOOL` (`TINYINT(1)`)	Ativo/inativo	—
`LONGTEXT`	Descrição do produto	Tamanho livre (limitado a 2.000 na API)

Capítulo 4

Tabelas

Campos, tipos, chaves e índices de cada tabela, gerados a partir dos modelos do Django e da migração do Flyway.

users · argo_identity

Contas de acesso de clientes, vendedores e administradores.

Campo	Tipo	Nulo	Chaves	Descrição
`id`	VARCHAR(36)	não	PK	UUID da conta; vira o sub do JWT
`name`	VARCHAR(120)	não		Nome exibido (pessoa ou loja)
`email`	VARCHAR(160)	não	UNIQUE	Login, sempre em minúsculas
`password_hash`	VARCHAR(128)	não		Hash bcrypt_sha256 do Django; a senha nunca é gravada
`role`	VARCHAR(12)	não		ADMIN, SELLER ou CUSTOMER
`active`	BOOL	não		Falso bloqueia login e refresh
`failed_attempts`	INTEGER	não		Senhas erradas seguidas
`locked_until`	DATETIME(6)	sim		Fim do bloqueio por tentativas
`last_login_at`	DATETIME(6)	sim		Último login bem-sucedido
`created_at`	DATETIME(6)	não		Criação
`updated_at`	DATETIME(6)	não		Última alteração

- CHECK ck_users_role: `role IN ('ADMIN', 'SELLER', 'CUSTOMER')`
- UNIQUE em `email`

refresh_tokens · argo_identity

Refresh tokens com rotação. Cada login abre uma família; cada troca cria um registro e revoga o anterior.

Campo	Tipo	Nulo	Chaves	Descrição
`id`	VARCHAR(36)	não	PK	UUID do registro
`user_id`	VARCHAR(36)	não	FK → users.id	Dono do token (ON DELETE CASCADE)
`family_id`	VARCHAR(36)	não		Sessão de origem; reuso revoga a família inteira

Campo	Tipo	Nulo	Chaves	Descrição
`token_hash`	VARCHAR(64)	não	UNIQUE	SHA-256 do token (o valor em si não é guardado)
`created_at`	DATETIME(6)	não		Emissão
`expires_at`	DATETIME(6)	não		Validade (7 dias)
`revoked_at`	DATETIME(6)	sim		Quando deixou de valer
`replaced_by`	VARCHAR(36)	sim		Registro que o substituiu na rotação

- Índice ix_refresh_tokens_family (family_id)
- Índice ix_refresh_tokens_user_id (user_id)

products · argo_catalog

Produtos de cada vendedor. Preço em centavos e estoque nunca negativo.

Campo	Tipo	Nulo	Chaves	Descrição
`id`	VARCHAR(36)	não	PK	UUID do produto
`seller_id`	VARCHAR(36)	não		UUID do vendedor (conta do serviço de identidade)
`seller_name`	VARCHAR(120)	não		Cópia do nome da loja
`sku`	VARCHAR(40)	não	UNIQUE	Código único, em maiúsculas
`name`	VARCHAR(140)	não		Nome
`description`	LONGTEXT	não		Descrição livre
`category`	VARCHAR(60)	não		Categoria (texto)
`price_cents`	INTEGER	não		Preço em centavos
`stock`	INTEGER	não		Saldo atual
`low_stock_threshold`	INTEGER	não		Limite de estoque baixo
`active`	BOOL	não		Falso tira da loja (exclusão lógica)
`version`	INTEGER	não		Concorrência otimista; sobe a cada alteração
`created_at`	DATETIME(6)	não		Criação
`updated_at`	DATETIME(6)	não		Última alteração

- CHECK ck_products_stock_non_negative: `stock >= 0`
- CHECK ck_products_price_positive: `price_cents > 0`

- Índice ix_products_seller (seller_id)
- Índice ix_products_category (category)

stock_movements · argo_catalog

Histórico imutável de toda alteração de estoque.

Campo	Tipo	Nulo	Chaves	Descrição
`id`	BIGINT AUTO_INCREMENT	não	PK	Sequencial
`product_id`	VARCHAR(36)	não	FK → products.id	Produto movimentado
`kind`	VARCHAR(10)	não		IN, OUT, COUNT, SALE ou RELEASE
`delta`	INTEGER	não		Variação com sinal
`stock_after`	INTEGER	não		Saldo depois do movimento
`reason`	VARCHAR(200)	não		Motivo
`order_id`	BIGINT	sim		Pedido de origem (vendas e devoluções)
`actor_id`	VARCHAR(36)	sim		Quem fez o ajuste manual
`created_at`	DATETIME(6)	não		Momento

- CHECK ck_stock_movements_kind: `kind IN ('IN', 'OUT', 'COUNT', 'SALE', 'RELEASE')`
- UNIQUE uq_stock_movements_order_product_kind: `(order_id, product_id, kind)`
- Índice ix_movements_product_created (product_id, created_at)

orders · argo_orders

Pedidos. Nome e e-mail do cliente são cópia do momento da compra.

Campo	Tipo	Nulo	Chaves	Descrição
`id`	BIGINT AUTO_INCREMENT	não	PK	Número interno; exibido como AR-000042
`customer_id`	VARCHAR(36)	não		UUID do cliente
`customer_name`	VARCHAR(120)	não		Cópia do nome
`customer_email`	VARCHAR(160)	não		Para o e-mail de confirmação
`status`	VARCHAR(12)	não		PENDING, CONFIRMED, REJECTED, SHIPPED, DELIVERED ou CANCELED
`total_cents`	INTEGER	não		Total em centavos

Campo	Tipo	Nulo	Chaves	Descrição
`items_count`	INTEGER	não		Soma das quantidades
`rejection_reason`	VARCHAR(300)	sim		Motivo da recusa
`invoice_number`	VARCHAR(30)	sim		Número da nota emitida pelo faturamento
`email_sent_at`	DATETIME(6)	sim		Envio do e-mail de confirmação
`created_at`	DATETIME(6)	não		Checkout
`updated_at`	DATETIME(6)	não		Última mudança

- CHECK ck_orders_status: `status IN ('PENDING', 'CONFIRMED', 'REJECTED', 'SHIPPED', 'DELIVERED', 'CANCELED')`
- CHECK ck_orders_total: `total_cents >= 0`
- Índice ix_orders_customer_created (customer_id, created_at)
- Índice ix_orders_status (status)

order_items · argo_orders

Linhas do pedido com nome e preço congelados.

Campo	Tipo	Nulo	Chaves	Descrição
`id`	BIGINT AUTO_INCREMENT	não	PK	Sequencial
`order_id`	BIGINT	não	FK → orders.id	Pedido (ON DELETE CASCADE)
`product_id`	VARCHAR(36)	não		Produto no catálogo
`seller_id`	VARCHAR(36)	não		Vendedor do item
`seller_name`	VARCHAR(120)	não		Cópia do nome da loja
`sku`	VARCHAR(40)	não		Cópia do SKU
`product_name`	VARCHAR(140)	não		Cópia do nome
`unit_price_cents`	INTEGER	não		Preço unitário no momento
`quantity`	INTEGER	não		Quantidade
`subtotal_cents`	INTEGER	não		Preço × quantidade

- CHECK ck_order_items_quantity: `quantity > 0`
- Índice ix_order_items_seller (seller_id)
- Índice ix_order_items_order (order_id)

order_status_history · argo_orders

Linha do tempo do pedido.

Campo	Tipo	Nulo	Chaves	Descrição
`id`	BIGINT AUTO_INCREMENT	não	PK	Sequencial
`order_id`	BIGINT	não	FK → orders.id	Pedido (ON DELETE CASCADE)
`status`	VARCHAR(12)	não		Situação no momento do evento
`note`	VARCHAR(300)	não		Texto exibido na linha do tempo
`actor`	VARCHAR(120)	não		Quem causou: cliente, loja, fila ou faturamento
`created_at`	DATETIME(6)	não		Momento

- Índice ix_order_status_history_order (order_id)

invoices · argo_billing

Notas fiscais emitidas pelo serviço Java. Uma por pedido.

Campo	Tipo	Nulo	Chaves	Descrição
`id`	BIGINT AUTO_INCREMENT	não	PK	Sequencial
`number`	VARCHAR(30)	não	UNIQUE	NF-ANO-000001 (contador invoice_number_seq)
`order_id`	BIGINT	não	UNIQUE	Pedido (único: emissão idempotente)
`order_code`	VARCHAR(20)	não		AR-000042
`customer_id`	VARCHAR(36)	não		Cliente
`customer_name`	VARCHAR(120)	não		Cópia do nome
`status`	VARCHAR(10)	não		ISSUED ou CANCELED
`total_cents`	BIGINT	não		Total
`tax_cents`	BIGINT	não		Valor aproximado dos tributos
`tax_rate`	DECIMAL(5,4)	não		Alíquota usada
`issued_at`	DATETIME(6)	não		Emissão
`canceled_at`	DATETIME(6)	sim		Cancelamento

- CHECK ck_invoices_status: `(status IN ('ISSUED', 'CANCELED'))`

- CHECK ck_invoices_total: `(total_cents >= 0 AND tax_cents >= 0)`
- Índice ix_invoices_customer (customer_id)

invoice_items · argo_billing

Itens da nota.

Campo	Tipo	Nulo	Chaves	Descrição
`id`	BIGINT AUTO_INCREMENT	não	PK	Sequencial
`invoice_id`	BIGINT	não	FK → invoices.id	Nota
`product_id`	VARCHAR(36)	não		Produto
`seller_id`	VARCHAR(36)	não		Vendedor (define quem vê a nota)
`description`	VARCHAR(140)	não		Descrição
`quantity`	INTEGER	não		Quantidade
`unit_price_cents`	BIGINT	não		Preço unitário
`total_cents`	BIGINT	não		Total da linha

- CHECK ck_invoice_items_quantity: `(quantity > 0)`
- Índice ix_invoice_items_invoice (invoice_id)
- Índice ix_invoice_items_seller (seller_id)

invoice_number_seq · argo_billing

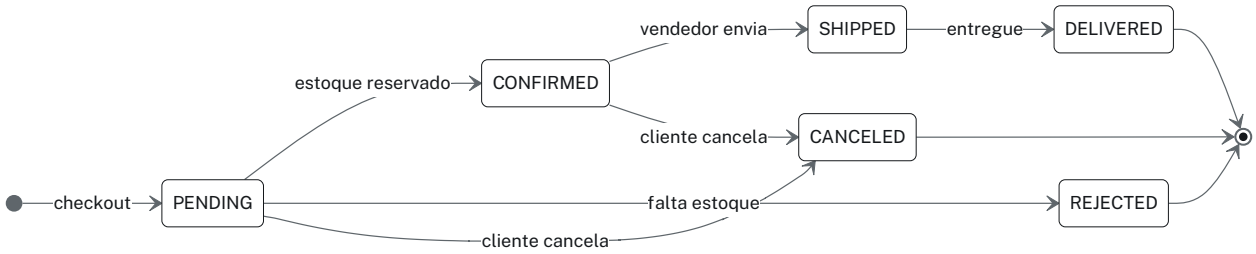
Contador da numeração das notas. O MySQL não tem SEQUENCE: a linha única é travada na emissão.

Campo	Tipo	Nulo	Chaves	Descrição
`id`	TINYINT	não	PK	Sempre 1
`next_val`	BIGINT	não		Próximo número de nota

- CHECK ck_invoice_number_seq_single: `(id = 1)`

Capítulo 5

Fluxo de situações do pedido



Transições válidas de `orders.status`

De	Para	Quem	Efeito no banco
—	`PENDING`	Checkout	Grava `orders`, `order_items` e o primeiro histórico na mesma transação
`PENDING`	`CONFIRMED`	Worker	Movimentos `SALE` em `stock_movements` e histórico
`PENDING`	`REJECTED`	Worker	`rejection_reason` preenchido; estoque intacto
`PENDING` ou `CONFIRMED`	`CANCELED`	Cliente ou admin	Movimentos `RELEASE` e nota cancelada (`invoices.status`)
`CONFIRMED`	`SHIPPED`	Vendedor ou admin	Histórico com rastreio
`SHIPPED`	`DELIVERED`	Vendedor ou admin	Histórico

Capítulo 6

Como parear o banco

Há três caminhos. Em todos, cada serviço só precisa da URL do próprio banco.

Com Docker (recomendado)

Nada a fazer. Na primeira subida, o MySQL executa `database/01-criar-bancos.sh`, que cria os quatro bancos em utf8mb4 e dá ao usuário `ARGO_DB_USER` acesso só a eles; cada serviço Django aplica as próprias migrações ao subir (`python manage.py serve`) e o Java roda o Flyway.

```
docker compose up -d
docker compose exec mysql sh -c 'mysql -u"$MYSQL_USER" -p"$MYSQL_PASSWORD" -e "SHOW DATABASES"'
docker compose exec catalog python manage.py showmigrations
```

MySQL próprio

1. Crie o usuário e os bancos:

```
CREATE USER 'argo'@'%' IDENTIFIED BY 'troque-esta-senha';
CREATE DATABASE argo_identity CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
CREATE DATABASE argo_catalog  CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
CREATE DATABASE argo_orders   CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
CREATE DATABASE argo_billing  CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
GRANT ALL PRIVILEGES ON argo_identity.* TO 'argo'@'%';
GRANT ALL PRIVILEGES ON argo_catalog.*  TO 'argo'@'%';
GRANT ALL PRIVILEGES ON argo_orders.*   TO 'argo'@'%';
GRANT ALL PRIVILEGES ON argo_billing.*  TO 'argo'@'%';
```

2. Aponte cada serviço:

```
identity  ARGO_SERVICE=identity ARGO_DATABASE_URL=mysql://argo:SENHA@HOST:3306/argo_identity
catalog   ARGO_SERVICE=catalog  ARGO_DATABASE_URL=mysql://argo:SENHA@HOST:3306/argo_catalog
orders    ARGO_SERVICE=orders   ARGO_DATABASE_URL=mysql://argo:SENHA@HOST:3306/argo_orders
billing   ARGO_BILLING_DB_URL=jdbc:mysql://HOST:3306/argo_billing ARGO_DB_USER=argo
ARGO_DB_PASSWORD=SENHA
```

3. As tabelas saem das migrações do Django na subida. Para criar antes, rode `python manage.py migrate` com as mesmas variáveis. Quem preferir o SQL de `database/argo_*.sql` (para revisão de um DBA, por exemplo) deve depois marcar as migrações com `python manage.py migrate --fake-initial`.
4. No `docker-compose.yml`, remova o serviço `mysql` e troque as URLs, ou rode os serviços fora do Docker.

Banco gerenciado

Amazon RDS for MySQL, Aurora MySQL, Google Cloud SQL e Azure Database for MySQL funcionam igual: crie os quatro bancos, libere o acesso pela rede dos serviços e use as URLs acima. Para exigir SSL,

acrescente `?ssl_mode=REQUIRED` nas URLs Python (o parâmetro vai para as `OPTIONS` do Django) e `?sslMode=REQUIRED` na URL JDBC.

Backup e restauração

```
for db in argo_identity argo_catalog argo_orders argo_billing; do
  docker compose exec -T mysql sh -c 'mysqldump -u"$MYSQL_USER" -p"$MYSQL_PASSWORD"
--single-transaction --no-tablespaces '"$db" > backup-$db.sql
done

docker compose exec -T mysql sh -c 'mysql -u"$MYSQL_USER" -p"$MYSQL_PASSWORD" argo_orders' <
backup-argo_orders.sql
```

Problemas comuns

Mensagem	O que fazer
`Unknown database 'argo_identity'`	O volume já existia antes do script de criação. Rode o `CREATE DATABASE` e o `GRANT` à mão ou `docker compose down -v`
`Access denied for user 'argo'`	Senha do `.env` diferente da usada na primeira subida; o MySQL guarda a senha inicial no volume
`Table 'users' already exists`	As tabelas foram criadas pelo SQL de `database/`; rode `python manage.py migrate --fake-initial`
`Schema-validation: missing table` (Java)	O Flyway não rodou; confira `ARGO_BILLING_DB_URL` e as permissões do usuário
`Public Key Retrieval is not allowed` (Java)	Conexão sem TLS com `caching_sha2_password`: use o TLS do servidor (padrão) ou acrescente `?allowPublicKeyRetrieval=true` na URL JDBC
Dados de demonstração não aparecem	Só são cadastrados em banco vazio e com `ARGO_SEED=true`

Capítulo 7

Evolução do esquema

1. **Serviços Python:** altere o modelo em `backend/argo/<serviço>/models.py` e gere a migração com `ARGO_SERVICE=all python manage.py makemigrations`. Ela é aplicada sozinha na próxima subida; o CI falha se um modelo mudar sem a migração correspondente.
2. Rode `python scripts/gerar_esquema.py` (com um MySQL acessível em `ARGO_ESQUEMA_DATABASE_URL`) para atualizar `database/argo_*.sql` e `docs/banco-de-dados.md` com o SQL que o próprio Django gera.
3. **Serviço Java:** nunca edite a V1. Crie `V2__descricao.sql` em `billing/src/main/resources/db/migration`; o Flyway aplica na próxima subida.
4. Mantenha as regras do capítulo 3: centavos, UTC, cópias do momento e restrições únicas para idempotência.
5. Rode os testes dos dois lados antes de publicar.

Capítulo 8

Mudanças da versão 1.1.0

Só o banco de pedidos (`argo_orders`) mudou. Os demais continuam iguais.

argo_orders

Tabela	Campo ou índice	Antes	Agora	Motivo
`orders`	`total_cents`	INTEGER	BIGINT	Pedidos acima de cerca de R$ 21 milhões estouravam o INTEGER
`order_items`	`subtotal_cents`	INTEGER	BIGINT	Preço máximo (R$ 1 milhão) vezes 99 unidades passava do limite
`orders`	`ix_orders_created` (`created_at`)	—	novo índice	Indicadores filtram pedidos pelo período

Os mesmos campos no faturamento (`invoices` e `invoice_items`) já eram BIGINT. `database/argo_orders.sql` e `docs/banco-de-dados.md` foram atualizados.

Como aplicar

Com Docker: nada a fazer. O comando `serve` roda a migração `orders.0002_totais_bigint_e_indice` ao subir.

MySQL próprio ou gerenciado: rode as migrações do Django com o serviço de pedidos:

```
ARGO_SERVICE=orders ARGO_DATABASE_URL=mysql://argo:SENHA@servidor:3306/argo_orders \
  python manage.py migrate
```

Ou, se as tabelas foram criadas à mão pelo SQL deste guia, aplique o equivalente:

```
USE argo_orders;
ALTER TABLE orders MODIFY total_cents bigint NOT NULL;
ALTER TABLE order_items MODIFY subtotal_cents bigint NOT NULL;
CREATE INDEX ix_orders_created ON orders (created_at);
```

Em tabelas grandes, o `ALTER TABLE` reescreve a tabela: rode fora do horário de pico.